

CARTLY ENGINEERING

API Reference

Mobile application, administration panel, backend services and Razorpay Test Mode.

Version: current `main` branch

Base path: `/api/v1`

Generated from: `backend/src` , `src/services` , and `admin/src`

1. Conventions

Base URL: configured by the client, ending in `/api/v1`. Successful JSON responses normally use `{ success: true, data: ... }`. Validation and application failures are handled by the central error middleware.

Access label	Requirement
Public	No access token.
Optional / guest	Bearer token when signed in, otherwise <code>x-guest-cart-token</code> .
User	<code>Authorization: Bearer <accessToken></code> .
Admin	Bearer token for a user with role <code>ADMIN</code> .

Common statuses: 200 success, 201 created, 204 success without a body, 400 validation, 401 unauthenticated, 403 forbidden, 404 missing resource, 409 conflict, 429 rate limited, and 500 server error.

2. System and authentication

GET

/api/health

PUBLIC

Service health probe. Alias: `GET /api/v1/health`.

POST

/auth/register

PUBLIC · OTP LIMITED

Create a customer and send email verification OTP.

Body: `name` 2–80, valid `email`, 10-digit `phone`, `password` 8–128.

POST

/auth/verify-email

PUBLIC

Verify registration. Body: `email`, six-digit `code`.

POST

/auth/resend-email-otp

PUBLIC · OTP LIMITED

Resend verification OTP. Body: `email`.

POST

/auth/login

PUBLIC

Authenticate and return access/refresh tokens. Body: `email`, `password`.

POST

/auth/refresh

PUBLIC

Rotate authentication using `refreshToken`.

POST

/auth/logout

PUBLIC

Revoke `refreshToken`. Returns 204.

POST

/auth/forgot-password

PUBLIC · OTP LIMITED

Send password-reset OTP. Body: `email`.

POST

/auth/reset-password

PUBLIC

Body: `email`, six-digit `code`, `newPassword` 8–128.

GET /auth/me

USER

Cartly API Reference · Generated 14 August 2026

Return the currently authenticated user. Used by mobile and admin clients.

3. Account and addresses

PATCH /account/profile

USER

Body: `name`, `phone` (10 digits or empty).

POST /account/change-password

USER

Body: `currentPassword`, `newPassword`.

DELETE /account

USER

Delete account after verifying body `password`. Returns 204.

GET /addresses

USER

List saved addresses.

POST /addresses

USER

Create address: `fullName`, `phone`, `addressLine`, `city`, `state`, `pincode`, `isDefault?`.

PATCH /addresses/:addressId

USER

Update address fields.

PATCH /addresses/:addressId/default

USER

Make address the default.

DELETE /addresses/:addressId

USER

Delete an owned address. Returns 204.

4. Catalogue and reviews

GET /categories

PUBLIC

List the public category tree/catalogue.

GET /products

PUBLIC

Search/filter products.

Query: `q`, `category`, `minPrice`, `maxPrice`, `minRating`, `inStock`, `onSale`, `sort` (popular/rating/price_asc/price_desc/newest), `page`, `limit` ≤ 50.

GET /products/:identifier

PUBLIC

Product details by ID or slug.

GET /products/:identifier/related

PUBLIC

Related products. Query: `limit` ≤ 20.

GET /products/:identifier/reviews

PUBLIC

Paginated reviews. Query: `page`, `limit` ≤ 50.

POST /products/:identifier/reviews

USER

Create/update review. Body: `rating` 1–5, optional `title`, optional `text`.

POST /products/:identifier/images

ADMIN

Multipart product-image upload; metadata supports `alt` and `position`.

DELETE /products/:identifier/images/:imageId

ADMIN

Delete a product image.

5. Cart and wishlist

POST /cart/guest

PUBLIC

Create a guest cart and return its token.

GET /cart

OPTIONAL / GUEST

Read current cart.

POST /cart/items

OPTIONAL / GUEST

Body: `variantId`, `quantity` 1–10.

PATCH /cart/items/:itemId

OPTIONAL / GUEST

Set `quantity` 1–10.

DELETE /cart/items/:itemId

OPTIONAL / GUEST

Remove one cart item.

DELETE /cart

OPTIONAL / GUEST

Clear the cart.

POST /cart/coupon

OPTIONAL / GUEST

Apply body `code`.

DELETE /cart/coupon

OPTIONAL / GUEST

Remove the active coupon.

POST /cart/merge

USER

Merge body `guestToken` into the user cart.

POST /cart/validate-checkout

USER

Validate inventory/pricing. Body `shippingMethod`: standard or express.

GET /wishlist

USER

List wishlist.

POST /wishlist

USER

Add body `productId`.

DELETE /wishlist/:productId

USER

Remove one product.

DELETE /wishlist

USER

Clear wishlist.

6. Orders, returns and fulfilment

POST /orders

USER

Create idempotent order.

Body: `idempotencyKey`, `shippingMethod`, `paymentMethod` (razorpay/cod), and `shippingAddress`.

GET /orders

USER

Paginated owned orders. Query: `page`, `limit` ≤ 50.

GET /orders/:orderId

USER

Owned order details.

GET /orders/:orderId/timeline

USER

Delivery/status timeline.

POST /fulfilment/orders/:id/cancel

USER

Body: `reason`; optional items array of `orderId`, `quantity`.

GET /fulfilment/orders/:id/return-eligibility

USER

Check whether owned order/items can be returned.

POST /fulfilment/orders/:id/returns

USER

Request return with `reason` and one or more `{orderId, quantity}`.

GET /fulfilment/returns

USER

List owned return/refund requests and timelines.

PATCH /fulfilment/returns/:id/pickup

USER

Set `pickupAt`.

PATCH /fulfilment/admin/orders/:id/status

ADMIN

Update status/message; optional carrier, tracking number, ETA and cancellation reason.

PATCH /fulfilment/admin/returns/:id/review

ADMIN

Cartly API Reference · Generated 14 August 2026

Body: approve , note .

POST /fulfilment/admin/returns/:id/complete

ADMIN

Complete return/refund workflow.

7. Payments and Razorpay Test Mode

POST /payments/initialize

USER

Create/reuse Razorpay order. Body: orderId , idempotencyKey .

POST /payments/verify

USER

Verify checkout signature. Body: local orderId , razorpay_order_id , razorpay_payment_id , 64-hex razorpay_signature .

POST /payments/failed

USER

Record checkout failure. Body: orderId ; optional Razorpay order ID, code and description.

POST /payments/orders/:orderId/refunds

ADMIN

Request refund. Body: optional amount , required reason , unique idempotencyKey .

POST /api/v1/payments/webhook

RAZORPAY SIGNATURE

Raw JSON webhook receiver. Validates x-razorpay-signature using RAZORPAY_WEBHOOK_SECRET .

External API calls made by the backend

Provider endpoint	Purpose
POST https://api.razorpay.com/v1/orders	Create test-mode payment orders.
POST https://api.razorpay.com/v1/payments/:paymentId/refund	Create refunds with Razorpay Basic authentication and idempotency protection in Cartly.

Security: only rzp_test_ keys are accepted. Key secret and webhook secret stay on the backend. Raw card numbers and CVV are not collected by Cartly.

8. Notifications and support

GET /notifications

USER

List notifications newest first.

PATCH /notifications/read-all

USER

Mark unread notifications read. Returns 204.

DELETE /notifications

USER

Delete all owned notifications. Returns 204.

POST /notifications/device-token

USER

Cartly API Reference · Generated 14 August 2026

Register push token. Body: token, platform android/ios.

GET /support/contacts

PUBLIC

Configured support email, phone and chat URL.

GET /support/tickets

USER

List owned support tickets.

POST /support/tickets

USER

Create ticket. Body: subject, message, optional channel APP/EMAIL/CHAT/CALL.

9. Administration API

Every route below requires an authenticated ADMIN.

GET /admin/dashboard

ADMIN

KPIs, charts, recent activity and stock alerts. Query: optional ISO from, to.

Categories

GET /admin/categories list · POST /admin/categories create (name, slug, parentId?) · PATCH /admin/categories/:id edit · DELETE /admin/categories/:id delete empty category.

Products and stock

GET /admin/products list with category, variants and images · POST /admin/products create product with at least one variant · PATCH /admin/products/:id edit product/images · DELETE /admin/products/:id soft-disable · PATCH /admin/variants/:id change price, old price or stock.

Product: name, slug, description, categoryId, active, variants[], images[]. Variant: sku, colour?, size?, price, oldPrice?, stock.
Image: url, alt?, position.

Orders and returns

GET /admin/orders · PATCH /admin/orders/:id/status · GET /admin/returns · PATCH /admin/returns/:id/review · PATCH /admin/returns/:id/status · POST /admin/returns/:id/complete.

Coupons

GET /admin/coupons · POST /admin/coupons · PATCH /admin/coupons/:id · DELETE /admin/coupons/:id. Fields: code, percentOff?, amountOff?, minimumCart?, active, startsAt?, expiresAt?.

Customers, reviews and reports

GET /admin/customers · PATCH /admin/customers/:id (active?, role?) · GET /admin/reviews · DELETE /admin/reviews/:id · GET /admin/reports/sales?from=&to=.

10. Client-to-API mapping

Client area	API modules used
Mobile authentication/account	auth, account, addresses
Shopping	catalogue, cart, wishlist, orders
After-sales	fulfilment, notifications, support
Checkout	cart validation, orders, payments, Razorpay Checkout SDK
Admin web	auth/me and all /admin routes; refund endpoint under /payments

Source of truth: Express route definitions and Zod schemas in `backend/src` . This document intentionally excludes database-internal operations and test-only requests.